

Week 10 reinforced learning

1. What is an agent, environment, state, action, policy, reward?

Agent is an entity that interacts with the environment with actions.

Environment is the entity where the agent interacts and receive feedback and rewards.

State describes the environment and the agent.

Action is a way the agent interacts with environment.

Policy determines the actions the agent takes. A rule to choose action given state,

Reward is a signal denoting how good an action was.

2. What is deterministic policy and stochastic policy?

Deterministic policy chooses the same action every time. The system always acts the same.

Stochastic can choose different actions each time. The system can act differently and explore more.

3. What changes state?

An action changes the state

4. What is the algorithm?

$S[0]$ = Initialize state

For i in range N :

$A = \text{action}(S[i])$

$S[i+1] = \text{new state}(S[i], a)$

5. What does a typical problem look like in RL?

Goal is to find the shortest path from start to finish

6. How can we adjust policy to get more rewards? What is the algorithm? What part can be stochastic?

We can adjust the policy over time to adapt better to the environment. The agent adjust the policy based on previous state, current state and reward received. We can add weights that we adjust to adjust the policy.

$S[0]$ = initial state

W = initial weights

For i in range N :

```

A = action(S[i],w)

S[i+1] = new state(S[i],a)

R = reward(S[i], S[i+1], a)

Update w

S[0] = <initial state>
w = <initial weights>
for i in range(N):
    a = action(S[i], w)      # POLICY
    S[i+1] = newstate(S[i], a) # ENVIRONMENT
    r = reward(S[i], S[i+1], a) # ENVIRONMENT
    <update w>               # POLICY LEARNING

```

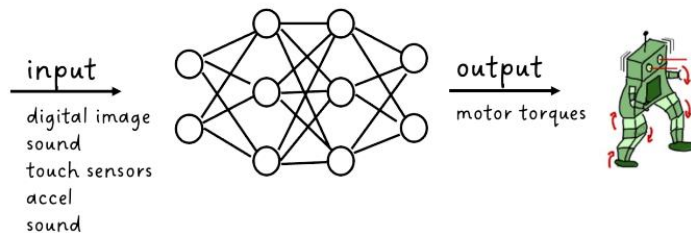
We must define how to take action/policy and how to update weights.

Every part can be stochastic (add randomness)

7. How can we represent a policy?

We can represent a policy with neural network or Q-table.

Representing the policy as a neural network



The weights in the network are given by the parameter vector w in $\text{action}(S,w)$

8. What are the three policies Q-table?

Q-table store all previous learning experience in a table with one entry for each (state, action) combination. We have three different policies Greedy, Epsilon-greedy, softmax, greedy always choose the highest reward, epsilon is a little more random, softmax is more random.

Softmax is a mathematical function used in machine learning and statistics to convert a vector of real numbers into a probability distribution. It takes as input a vector of arbitrary real-valued scores

and produces a probability distribution over the scores such that the sum of the probabilities of all possible outcomes is equal to one.

	a_1	a_2	a_3		a_m
s_1					
s_2					
.					
.					
s_m					

Q-table

average reward obtained after taking this action when in this state

Three different ways of using the Q-table

greedy: $\text{action} = \underset{a}{\operatorname{argmax}} Q[s,a]$

ϵ -greedy: $\text{action} = \begin{cases} \underset{a}{\operatorname{argmax}} Q[s,a], & \text{with prob } 1 - \epsilon \\ \text{random action}, & \text{with prob } \epsilon \end{cases}$

softmax: $\text{action} = \begin{cases} a_1 & \text{with prob } \frac{e^{Q[s,a_1]/\tau}}{\sum e^{Q[s,a_k]/\tau}} \\ \dots & \dots \\ a_m & \text{with prob } \frac{e^{Q[s,a_m]/\tau}}{\sum e^{Q[s,a_k]/\tau}} \end{cases}$

An example

Suppose we are in state s_2 .
How do we choose the next action?



	a_1	a_2	a_3
s_1	5	-1	3
s_2	0	2	11
s_3	7	9	-5
s_m	8	2	0

greedy: choose a_3

ϵ -greedy: Draw a random number $X \sim U[0,1]$

If $X \geq \epsilon$: choose a_3

else: select action at random

softmax: Draw $(X_1, X_2, X_3) \sim \text{multinomial}(1; p_1, p_2, p_3)$

If $X_1 = 1$: choose a_1

else if $X_2 = 1$: choose a_2

else if $X_3 = 1$: choose a_3

$$p_1 = \frac{\exp(0)}{\exp(0) + \exp(2) + \exp(11)}$$

$$p_2 = \frac{\exp(2)}{\exp(0) + \exp(2) + \exp(11)}$$

$$p_3 = \frac{\exp(11)}{\exp(0) + \exp(2) + \exp(11)}$$

9. How can weights be initialized?

It can be initialized or from previous learning cycle.

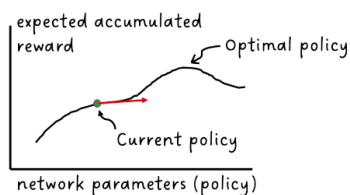
10. What is policy-gradient methods and actor-critic methods?

Policy-gradient methods and actor-critic methods is methods for policies implemented with neural networks.

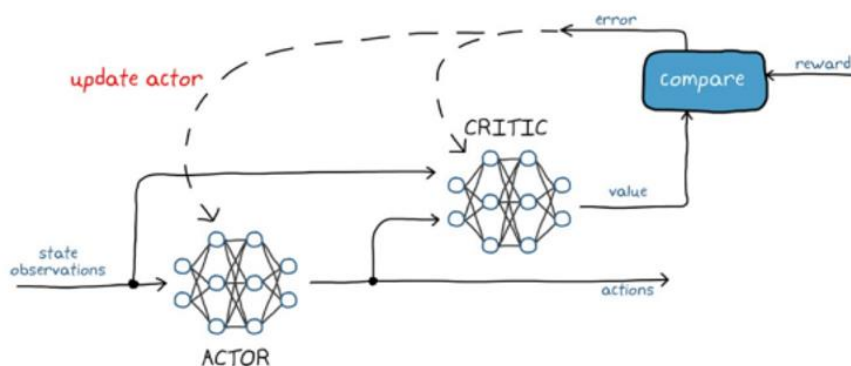
Policy-gradient methods executes current policy, collect rewards, after several iterations adjust the weights in the direction of increased expected reward.

Policy-gradient methods

- Execute current policy
- Collect rewards
- After several iterations adjust the weights in the direction of increased expected reward



actor-critic methods have two neural nets interact and are trained together. The critic network learns the values of different (state, action) pairs from received rewards. The actor network learns correct actions using feedback from critic network.



11. What are the problems with Q-table? And how do we fix them?

Starting point: $Q[s_i, a_j] = (r_1 + r_2 \dots + r_k) / k$

Problem 1: the environment may change over time and in that case we would trust recent rewards more than older ones

Solution: Suppose we are in state s , current policy take action a , and the reward is r . We put slightly more weight on newer solutions. we let:

$$Q[s, a] = (1-\mu)Q[s, a] + \mu r$$

Where μ is typically 0.1 or smaller

Problem 2: the update rule only considers the reward for the next move. We want $Q[s, a]$ to reflect the total accumulated reward from s and all the way to the final destination.

Solution: Suppose we are in state s , policy take action a , and the reward is r , the new state is s' , and policy says we should then take action a'

$$Q[s, a] = (1-\mu)Q[s, a] + \mu(r + \gamma Q[s', a'])$$

$Q[s', a']$ depends on policy, if greedy then we take max

$$Q[s, a] = (1-\mu)Q[s, a] + \mu (r + \gamma Q[s', a'])$$

This learning rule is called SARSA.

Discount rate - a number $\gamma \in (0,1)$ close to 1

12. What is SARSA?

SARSA is a learning rule to update $Q[s, a]$. it takes into account all future rewards.

13. What is discounted future reward?

The discounted future reward in SARSA refers to the sum of the expected rewards that an agent can obtain in the future by following a particular policy. It is calculated as the sum of the rewards obtained at each time step, weighted by a discount factor, which determines the importance of future rewards relative to immediate rewards.

The expression tells us that rewards received later on the path to the destination are reduced because there is some uncertainty in each action we take and that uncertainty accumulates along the path.

$$Q[s,a] = r + \gamma r' + \gamma^2 r'' + \gamma^3 r''' + \dots$$

14. What is Q-learning? What are the differences between SARSA and Q-learning?

In Q-learning we replace the term $Q[s', a']$ in SARSA with maximal value of $Q[s', :]$

$$Q[s,a] = (1-\mu)Q[s,a] + \mu (r + \gamma \max Q[s', :])$$

If policy is greedy then Q-learning and SARSA are identical

15. What is the difference between on- and off-policy? Categorize SARSA and Q-learning.

Off-policy: when we learn, we don't follow the policy the agent has, but make other assumptions. Do not assume we act according to policy. In Q-learning we do not act according to policy in $\max Q[s', :]$ that estimate optimal future value. This assumption follows greedy, but it may not be the same policy as agent.

On-policy: we follow the same policy as agent in learning. In SARSA $Q[s', a']$ follows the same policy as agent.

Q-learning is off-policy while SARSA is on-policy.

Off-Policy Learning

On-Policy Learning

The Q-Learning Algorithm

- **Initialisation**
 - set $Q(s, a)$ to small random values for all s and a
- **Repeat:**
 - initialise s
 - repeat:
 - * select action a using ϵ -greedy or another policy
 - * take action a and receive reward r
 - * sample new state s'
 - * update $Q(s, a) \leftarrow Q(s, a) + \mu(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$
 - * set $s \leftarrow s'$
 - For each step of the current episode
- Until there are no more episodes

The Sarsa Algorithm

- **Initialisation**
 - set $Q(s, a)$ to small random values for all s and a
- **Repeat:**
 - initialise s
 - choose action a using the current policy
 - repeat:
 - * take action a and receive reward r
 - * sample new state s'
 - * choose action a' using the current policy
 - * update $Q(s, a) \leftarrow Q(s, a) + \mu(r + \gamma Q(s', a') - Q(s, a))$
 - * $s \leftarrow s', a \leftarrow a'$
 - for each step of the current episode
- Until there are no more episodes

Source: Marsland



68

On-policy vs off-policy learning

- Q-learning (off-policy):

$$Q(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\mu}_{\text{learning rate}} \cdot \left(\underbrace{r_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\substack{\text{estimate of optimal future value} \\ \text{learned value}}} - \underbrace{Q(s_t, a_t)}_{\text{old value}} \right)$$

- Sarsa (on-policy):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \mu[r_{t+1} + \gamma \underbrace{Q(s_{t+1}, a_{t+1})}_{\text{learned value}} - Q(s_t, a_t)]$$

Depending on the policy, Q-learning is more risky than SARSA. If it is epsilon-greedy, SARSA can choose random actions and can take wrong turn and end up with bad reward. Therefore, it chooses safe actions (steps) so that it does not make bad action on accident. But since Q-learning chooses the max value in learning, it can be more risky because there is no randomness.

16. Hvorfor har vi discount factor?

Discount factor = 1 betyr at fremtiden er sikker

The discount factor reduces the impact of future expected rewards when calculating the q-value of the current state. (1p) It is important to have one when the future is a bit uncertain, since when we

have some uncertainty, immediate rewards are more valuable. (1p) If we reduced the impact factor, states where we have high expected future rewards would be valued a bit less. In the example above, the only situation where we have a higher than 0 expected future reward is when going left in state 3. With a discount factor of 0.1, we would then end up with a negative q-value despite there being a nice future reward waiting in state 2. (1p) With a discount factor of 1, we are assuming that the future is completely certain, or at least that we don't care about any uncertainty about the future in our calculations. (1p)